

Алгоритм Эдмондса-Карпа. Алгоритм масштабирования потока

Подготовил Сивухин Никита. По вопросам пишите на почту sivukhin.work+teach@gmail.com

Метод Форда-Фалкерсона не определяет то, вдоль какого дополняющего пути нужно пропускать поток в остаточной сети на очередной итерации. Если выбирать произвольный путь, то алгоритм будет работать за $O(E|f|)$ в худшем случае. Однако, если выбирать пути особым образом можно добиться лучшей асимптотики.

Суть алгоритма Эдмондса-Карпа заключается в том, чтобы на каждой итерации пропускать поток вдоль кратчайшего по рёбрам пути в остаточной сети. Важно также пропускать максимально возможную величину потока $c_f(p)$ вдоль кратчайшего пути p , т.е. $c_f(p) = \min\{c_f(u, v) \mid (u, v) \in p\}$.

```
1 struct Edge { int to, flow, capacity; };
2 struct Graph {
3     int n, S, T;
4     vector<Edge> edges;
5     vector<vector<int>> graph;
6     ...
7     int max_flow() {
8         int total_flow = 0;
9         while (true) {
10             int flow = bfs();
11             if (flow == 0) { break; }
12             total_flow += flow;
13         }
14         return total_flow;
15     }
16 };
```

Для доказательства времени работы такого подхода докажем сначала следующее вспомогательное утверждение:

Лемма 1. Если в транспортной сети $G = (V, E)$ увеличение потока происходит вдоль кратчайшего $s \rightsquigarrow t$ пути в остаточной сети G_f , то для всех вершин $v \in V \setminus \{s, t\}$ длина кратчайшего пути $d_f(s, v)$ в остаточной сети G_f не уменьшается.

Доказательство. От противного: допустим расстояние до некоторых вершин уменьшилось. Рассмотрим среди них такую вершину v , что расстояние от s в G'_f до неё минимально, т.е. $d'_f(s, v) < d_f(s, v)$ и $d'_f(s, v)$ минимально возможное среди всех таких вершин, где d'_f — это расстояние в остаточной сети G'_f после увеличения потока вдоль кратчайшего пути в G_f .

Рассмотрим кратчайший путь p из s в v в G'_f и обозначим вершину предшествующую v в этом пути за u . Ясно, что $d'_f(s, u) + 1 = d'_f(s, v)$. Отсюда следует, что $d'_f(s, u) \geq d_f(s, u)$ так как иначе вершина v не являлась бы ближайшей вершиной к s для которой уменьшилось расстояние в остаточной сети.

Заметим, что ориентированное ребро (u, v) не может принадлежать остаточной сети G_f , потому что отсюда бы следовало, что $d_f(s, v) \leq d_f(s, u) + 1 \leq d'_f(s, u) + 1 = d'_f(s, v)$, что противоречит выбору вершины v как вершины с уменьшившимся расстоянием в остаточной сети.

Значит $(u, v) \notin E_f$, но известно, что $(u, v) \in E'_f$, а значит увеличение потока произошло по обратному ребру (v, u) . Значит, дополняющий путь для увеличения потока проходил через ребро (v, u) : $s \rightsquigarrow v \rightarrow u \rightsquigarrow t$. Отсюда следует, что $d_f(s, v) = d_f(s, u) - 1 \leq d'_f(s, u) - 1 = d'_f(s, v) - 2 \Rightarrow d'_f(s, v) \geq d_f(s, v) + 2$, что опять противоречит выбору вершины v . $\square$

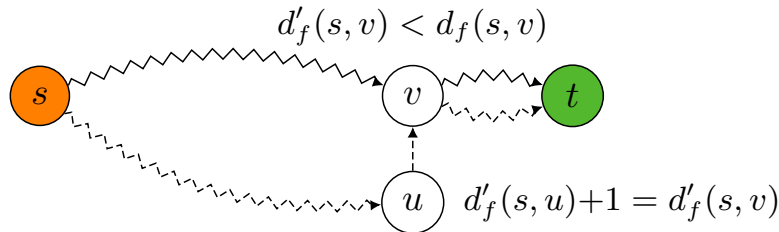


Рис. 1: Иллюстрация к доказательству: сплошные линии изображают пути в графе G_f , а пунктирные — в G'_f

Заметим, что данная лемма напоминает лемму из лекции про алгоритма Хопкрофта-Карпа, которая показывает, что длина M' -чередующейся цепи не уменьшается, если на каждом шаге производить чередование вдоль кратчайшей цепи.

Будем считать, что ребро (u, v) является *критическим* на очередной итерации алгоритма Эдмондса-Карпа, если оно принадлежит кратчайшему пути на текущей итерации, а также его остаточная пропускная способность равна изменению потока вдоль пути p , т.е. $(u, v) \in p \wedge c_f(u, v) = c_f(p)$.

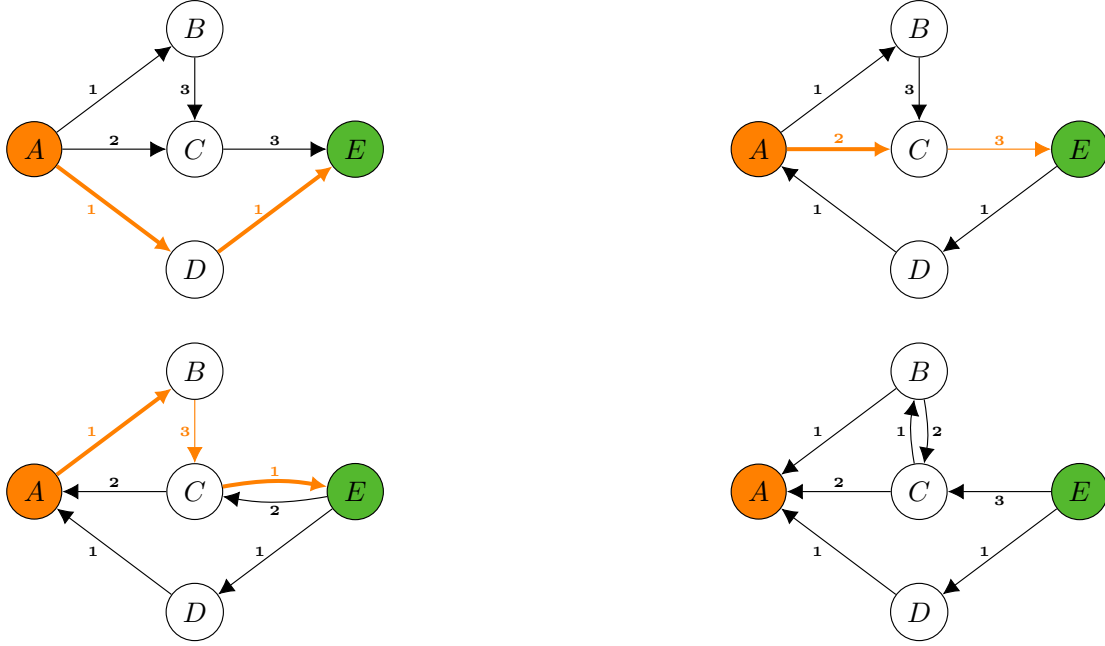


Рис. 2: Пример работы алгоритма Эдмондса-Карпа. «Толстые» рёбра обозначают «критическое» ребро на очередной итерации

Покажем, что каждое ребро (u, v) может быть *критическим* не более чем в $O(|V|)$ итерациях алгоритма. Для этого заметим, что если ребро (u, v) было *критическим* в сети G_f , то на следующей итерации оно пропадет из остаточной сети, т.к. величина потока вдоль этого ребра будет совпадать с его пропускной способностью. Тогда, чтобы ребро опять появилось в остаточной сети и могло стать *критическим* снова необходимо, чтобы дополняющий путь прошёл через обратное ребро (v, u) . Таким образом, если ребро (u, v) являлось критическим в остаточной сети G_f , а (v, u) — принадлежало кратчайшему пути в сети G_f^* , то отсюда следует, что $d_f(s, v) = d_f(s, u) + 1$ и $d_f^*(s, u) = d_f^*(s, v) + 1$, а так как $d_f^*(s, v) \geq d_f(s, v)$ из леммы 1, то $d_f^*(s, u) \geq d_f(s, u) + 2$. Так как кратчайшее расстояние ограничено количеством вершин $|V|$, то отсюда следует, что ребро не может быть *критическим* более чем на $\frac{|V|}{2}$ итерациях.



Рис. 3: Чтобы ребро стало критическим снова необходимо, чтобы увеличение потока прошло через обратное ребро

Осталось заметить, что на каждой итерации есть хотя бы одно *критическое* ребро (так как $c_f(p) = \min\{c(u, v) \mid (u, v) \in P\}$, то есть хотя бы одно ребро где достигается этот минимум), а значит суммарное число итераций алгоритма не превосходит $O(|V||E|)$.

Так как BFS для поиска кратчайшего пути по рёбрам работает за время $O(|E|)$, то суммарная асимптотика алгоритма равна $O(|V||E|^2)$.

Масштабирование потока

Логично, что на очередной итерации выгоднее находить дополняющие пути с наибольшей пропускной способностью $c_f(p)$. Можно формализовать эту идею и получить нетривиальное ускорение относительно разобранного ранее алгоритма Эдмонса-Карпа.

Для описания алгоритма масштабирования потока введём дополнительный параметр Δ , и на его основе построим вспомогательную остаточную сеть, состоящую из рёбер с остаточной пропускной способностью не меньше чем Δ , т.е. $G_{f,\Delta} = (V, \{e \mid e \in E_f \wedge c_f(e) \geq \Delta\})$.

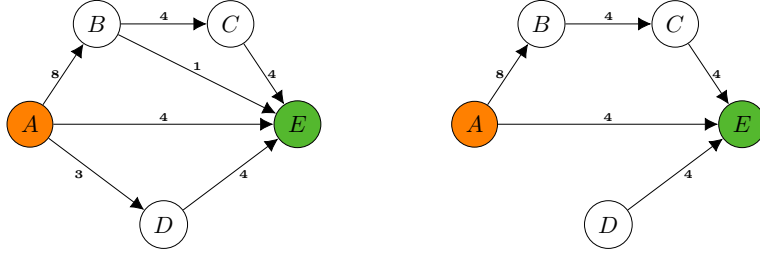


Рис. 4: Пример транспортной сети и вспомогательной остаточной сети с $\Delta = 4$

Алгоритм масштабирования потока начинает свою работу с $\Delta = \max_{e \in E} \{c(e)\}$ и находит дополняющие пути в вспомогательной остаточной сети $G_{f,\Delta}$ как алгоритм Форда-Фалкерсона. Когда дополняющих путей в $G_{f,\Delta}$ не осталось — алгоритм уменьшает Δ в 2 раза и повторяет весь процесс до тех пор, пока Δ не станет равна 1 (в случае чего алгоритм вырождается в алгоритм Форда-Фалкерсона).

```

1 struct Edge { int to, flow, capacity; };
2 struct Graph {
3     int n, S, T;
4     vector<Edge> edges;
5     vector<vector<int>> graph;
6     ...
7     int max_flow() {
8         int delta = std::max_element(edges.begin(), edges.end(),
9             [](const Edge &a, const Edge &b) { return a.capacity < b.capacity; })
10            ->capacity;
11         int total_flow = 0;
12         while (delta >= 1) {
13             while (true) {
14                 int flow = dfs(delta);
15                 if (flow == 0) { break; }
16                 total_flow += flow;
17             }
18             delta /= 2;
19         }
20         return total_flow;
21     }
22 };

```

Для оценки времени работы этого алгоритма докажем вспомогательную лемму:

Лемма 2. Если величина потока в конце итерации алгоритма с параметром Δ равна $|f_\Delta|$, то максимальный поток в оригинальной сети строго меньше $|f_\Delta| + \Delta|E|$.

Доказательство. Построим разрез $\langle S, T \rangle$ на основе вспомогательной остаточной сети $G_{f,\Delta}$ в конце очередной итерации алгоритма, когда в сети $G_{f,\Delta}$ не осталось дополняющих путей (в долю S отнесём все вершины, достижимые из истока s по рёбрам в $G_{f,\Delta}$). Заметим, что все рёбра между долями $S \rightarrow T$ имеют остаточную пропускную способность строго меньше Δ (иначе, это ребро было бы в $G_{f,\Delta}$ и мы должны были отнести правую вершину в долю S), а значит величина такого разреза строго меньше $\Delta|E|$. Так как величина любого потока не превосходит величины любого разреза, значит максимальный поток в остаточной сети в конце итерации строго меньше $\Delta|E|$, а значит максимальный поток в оригинальном графе строго меньше $|f_\Delta| + \Delta|E|$. $\square$

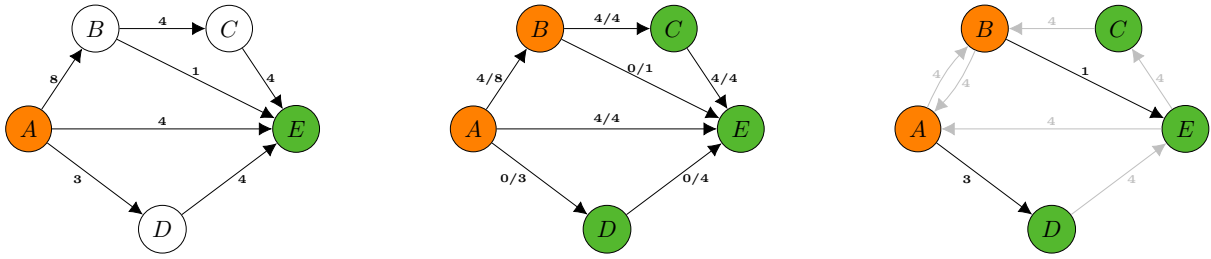


Рис. 5: Пример работы алгоритма масштабирования потока при $\Delta=4$. После завершения итерации разрез $\langle \{A, B\}, \{C, D, E\} \rangle$ состоит из рёбер e таких, что $c_f(e) < \Delta$.

Из леммы 2 следует, что для очередного значения Δ алгоритм найдет менее $2E$ дополняющих путей, т.к. каждый путь имеет пропускную способность хотя бы Δ , а максимальный поток остаточной сети с предыдущей итерации строго меньше чем $2\Delta E$.

Таким образом, алгоритм совершит $O(E \log U)$ (где $U = \max\{c(e) \mid e \in E\}$) операций нахождения дополняющей цепи, каждая из которых работает за $O(|E|)$, откуда следует что общая асимптотика алгоритма равна $O(|E|^2 \log U)$.